LIBXS

LIBXS is a portable C library providing building blocks for memory operations, numerics, synchronization, and more -- with a focus on performance and minimal dependencies. Targets x86-64, AArch64, and RISC-V; requires only a C89 compiler. Originally developed as part of LIBXSMM.

Functionality

| Domain | Header | Description | |||| | Memory | `libxs_mem.h` | Comparison, hashing (CRC32), matrix copy/transpose, shuffle | | GEMM | `libxs_gemm.h` | Batched dense GEMM (strided, pointer-array, grouped) | | Math | `libxs_math.h` | Matrix comparison, GCD/LCM, coprime, BF16 conversion | | MHD | `libxs_mhd.h` | Read/write MetaImage (MHD/MHA) files | | Histogram | `libxs_hist.h` | Thread-safe histogram with running statistics | | Malloc | `libxs_malloc.h` | Pool-based allocator (steady-state, no system calls) | | RNG | `libxs_rng.h` | Thread-safe pseudo-random number generation (SplitMix64) | | Sync | `libxs_sync.h` | Portable atomics, locks, TLS, and file locking | | Timer | `libxs_timer.h` | High-resolution timing via calibrated TSC | | CPUID | `libxs_cpuid.h` | CPU feature detection (SSE to AVX-512, AArch64, RISC-V) | | Registry | `libxs_reg.h` | Thread-safe key-value store with per-thread caching | | Utils | `libxs_utils.h` | ISA feature gates, bit-scan, SIMD helpers |

See also: Fortran Interface, Scripts.

Build

```
make GNU=1
```

The library is compiled for SSE4.2 by default but dynamically dispatches to the best ISA available at runtime (up to AVX-512). Use `SSE=0` to compile natively for the build host.

| Variable | Default | Description | |||| | GNU | 0 | Use GNU GCC-compatible compiler | | DBG | 0 | Debug build | | SYM | 0 | Include debug symbols (-g) | | SSE | 1 | x86 baseline: 0=native, 1=SSE4.2 (portable) |

CMake is also supported (header-only or library):

```
cmake -S . -B build -DLIBXS_HEADER_ONLY=ON
cmake --build build
```

Installation

Install into a chosen prefix:

```
make GNU=1 -j $(nproc) install PREFIX=$HOME/libxs
```

This installs headers, the Fortran module, the static and shared libraries, and the header-only source tree under `PREFIX`.

Out-of-tree builds are also supported:

```
mkdir /tmp/libxs-build && cd /tmp/libxs-build
make -j $(nproc) -f /path/to/libxs/Makefile
```

Usage

Library -- link against `libxs.a` (or `.so`) and include the desired headers:

```
#include <libxs_mem.h>
#include <libxs_timer.h>
```

Header-only (explicit) -- include `libxs_source.h` (no separate library needed). Safe to include from multiple translation units:

```
#include <libxs_source.h>
```

Header-only (implicit) -- compile with `-DLIBXS_SOURCE` and any LIBXS public header automatically includes the implementation. No special include order is required. When used through LIBXSTREAM without a pre-built library (`-DLIBXSTREAM_SOURCE`), `LIBXS_SOURCE` is implied automatically.

Fortran -- use the provided module (documentation):

```
USE :: libxs, ONLY: libxs_memcmp
```

Samples

Sample	Description
gemm	Batched DGEMM (strided, pointer-array, grouped) with OMP
memory	Benchmarks for comparison, matrix copy, and transpose
ozaki	Ozaki-scheme low-precision GEMM with intercepted BLAS
registry	Registry dispatch microbenchmark
scratch	Pool allocator vs system malloc
shuffle	Shuffling strategies comparison
sync	Lock implementation microbenchmarks

License

BSD 3-Clause

LIBXS Domains

CPU Identification

Header: `libxs_cpuid.h`

Portable CPU feature detection for x86-64, AArch64, and RISC-V targets. Returns an ISA level that can be compared numerically -- higher values indicate more capable instruction sets. x86 levels use thermometer ordering: higher numeric value implies all features of lower levels. AVX10/256 sits below AVX512 because it lacks 512-bit vectors. On Linux, AMX tile state (XTILE_DATA) is automatically requested via `arch_prctl` when AMX capability is detected.

ISA Constants

Constant	Value	Architecture
`LIBXS_TARGET_ARCH_UNKNOWN`	0	Unknown / unsupported
`LIBXS_TARGET_ARCH_GENERIC`	1	Portable scalar baseline
`LIBXS_X86_GENERIC`	1002	x86-64 baseline
`LIBXS_X86_SSE3`	1003	SSE3
`LIBXS_X86_SSE42`	1004	SSE4.2
`LIBXS_X86_AVX`	1005	AVX
`LIBXS_X86_AVX2`	1006	AVX2 + FMA
`LIBXS_X86_AVX10_256`	1030	AVX10.1/256: all features, 256-bit max (Sierra Forest)
`LIBXS_X86_AVX512`	1100	AVX-512 (F + CD + DQ + BW + VL + VNNI)
`LIBXS_X86_AVX512_AMX`	1105	AVX-512 + AMX (TILE + INT8 + BF16); Sapphire/Granite Rapids
`LIBXS_X86_AVX512_INT8`	1110	AVX-512 + AVX-VNNI-INT8 (VPDPBUUD/BSSD)
`LIBXS_X86_AVX10_512`	1200	AVX10.1/512 + AMX + INT8: full feature set
`LIBXS_AARCH64`	2001	ARMv8.1 baseline
`LIBXS_AARCH64_SVE128`	2201	SVE 128-bit
`LIBXS_AARCH64_SVE256`	2301	SVE 256-bit
`LIBXS_AARCH64_SVE512`	2401	SVE 512-bit
`LIBXS_RV64_MVL128`	3001	RISC-V RVV 128-bit
`LIBXS_RV64_MVL256`	3002	RISC-V RVV 256-bit
`LIBXS_RV64_MVL128_LMUL`	3003	RISC-V RVV 128-bit (non-unit LMUL)
`LIBXS_RV64_MVL256_LMUL`	3004	RISC-V RVV 256-bit (non-unit LMUL)
`LIBXS_X86_ALLFEAT`	1999	x86 sentinel (all features)
`LIBXS_AARCH64_ALLFEAT`	2999	AArch64 sentinel (all features)
`LIBXS_RV64_ALLFEAT`	3999	RISC-V sentinel (all features)

Types

```
typedef struct libxs_cpuid_t {
  char model[256];   /* CPU model name from OS */
  int constant_tsc;  /* non-zero if TSC is invariant */
} libxs_cpuid_t;
```

Functions

```
int libxs_cpuid(libxs_cpuid_t* info);
```

Detect the ISA level of the current platform. Optionally fills `info` with the CPU model name and TSC capability. Returns a constant from the table above.

```
const char* libxs_cpuid_name(int id);
```

Returns a human-readable string for the given ISA constant, e.g., `"avx2"`.

```
int libxs_cpuid_id(const char* name);
```

Translates a name (e.g., `"avx512"`) to the corresponding ISA constant.

```
int libxs_cpuid_vlen(int id);
```

Returns the SIMD vector length in bytes for the given ISA level (0 for scalar-only targets).

project: LIBXS author: Intel Corporation license: BSD 3-Clause License summary: Small utility library. project_github: https://github.com/hfp/libxs/ project_download: https://github.com/hfp/libxs/releases/latest/ favicon: ../.theme/img/favicon.png css: ../.theme/ford.css output_dir: fortran src_dir: ../include search: true

Reference Documentation

GEMM: Matrix Multiplication

Header: `libxs_gemm.h`

Batched general matrix-matrix multiplication (GEMM). Operations are expressed as C := alpha * op(A) * op(B) + beta * C, where op() is an optional transpose. The caller can supply MKL JIT, LIBXSMM, or BLAS kernels; otherwise a built-in default kernel (auto-vectorized) is used.

Types

```
typedef void (*libxs_gemm_dblas_t)(
  const char* transa, const char* transb,
  const int* m, const int* n, const int* k,
  const double* alpha, const double* a, const int* lda,
                       const double* b, const int* ldb,
  const double* beta,        double* c, const int* ldc);
```

Standard Fortran BLAS dgemm signature (e.g., `dgemm_`).

```
typedef void (*libxs_gemm_sblas_t)(
  const char* transa, const char* transb,
  const int* m, const int* n, const int* k,
  const float* alpha, const float* a, const int* lda,
                      const float* b, const int* ldb,
  const float* beta,        float* c, const int* ldc);
```

Standard Fortran BLAS sgemm signature (e.g., `sgemm_`).

```
typedef void (*libxs_gemm_djit_t)(void* jitter,
  const double* a, const double* b, double* c);
typedef void (*libxs_gemm_sjit_t)(void* jitter,
  const float* a, const float* b, float* c);
```

MKL JIT kernel signatures. Shape, alpha, beta, and transpose info are baked into the jitter handle.

```
typedef struct libxs_gemm_param_t {
  void* op[4]; const void* a[6]; const void* b[6]; void* c[6];
} libxs_gemm_param_t;
```

XGEMM parameter struct, layout-compatible with `libxsmm_gemm_param`. Only `a[0]` (a.primary), `b[0]` (b.primary), and `c[0]` (c.primary) are used for plain GEMM.

```
typedef void (*libxs_gemm_xfn_t)(const void*);
```

Opaque XGEMM kernel signature: `void(const libxs_gemm_param_t*)`.

```
typedef enum libxs_gemm_flags_t {
  LIBXS_GEMM_FLAGS_DEFAULT = 0,
  LIBXS_GEMM_FLAG_NOLOCK = 1
} libxs_gemm_flags_t;
```

Flags controlling batch synchronization. By default, `_task` variants synchronize C-matrix updates. Set `LIBXS_GEMM_FLAG_NOLOCK` when no duplicate C pointers exist.

```
typedef struct libxs_gemm_shape_t {
  libxs_data_t datatype;
  int transa, transb;
  int m, n, k, lda, ldb, ldc;
  double alpha, beta;
} libxs_gemm_shape_t;
```

GEMM shape: problem geometry, transpose flags, and scalar coefficients. Alpha and beta are stored as double regardless of datatype (float values are promoted without loss). Also serves as registry key when caching dispatched configurations (all-int fields avoid padding).

```
typedef struct libxs_gemm_config_t {
  libxs_gemm_dblas_t dgemm_blas;
  libxs_gemm_sblas_t sgemm_blas;
  libxs_gemm_djit_t dgemm_jit;
  libxs_gemm_sjit_t sgemm_jit;
  libxs_gemm_xfn_t xgemm;
  void* jitter;
  libxs_gemm_flags_t flags;
  libxs_gemm_shape_t shape;
} libxs_gemm_config_t;
```

Configuration supplying GEMM kernels. Pass NULL to batch functions to use the built-in default kernel (auto-vectorized, no BLAS dependency). Kernel selection priority:

1. JIT kernel (`dgemm_jit`/`sgemm_jit` + `jitter`) if non-NULL,
2. XGEMM kernel (`xgemm`) if non-NULL,
3. BLAS kernel (`dgemm_blas`/`sgemm_blas`) if non-NULL,
4. built-in default kernel.

Only the function pointers matching the datatype need to be set. The `shape` member is populated by `libxs_gemm_dispatch`.

Functions

```
int libxs_gemm_dispatch(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta,
  void* registry /* = NULL */);
```

Dispatch a GEMM kernel and populate `config` accordingly. With MKL JIT (highest priority): sets `config->dgemm_jit` or `sgemm_jit` + `jitter`. With LIBXSMM (fallback): sets `config->xgemm` via `libxsmm_dispatch_gemm`. Without either: config is unchanged (falls through to BLAS/default). The caller should memset config to zero before the first call. Returns `EXIT_SUCCESS` on success, `EXIT_FAILURE` when no JIT backend is available. If `registry` is non-NULL, dispatched configs are cached by shape: a hit copies the cached config, a miss dispatches and stores.

```
int libxs_gemm_ready(const libxs_gemm_config_t* config);
```

Check whether a dispatched config holds a usable kernel. Returns nonzero if `libxs_gemm_call` would succeed, zero otherwise.

```
int libxs_gemm_call(
  const libxs_gemm_config_t* config,
  const void* a, const void* b, void* c);
```

Call the GEMM kernel previously dispatched into config. Follows the documented priority (JIT > XGEMM > BLAS fallback). Returns `EXIT_SUCCESS` if a kernel was called, `EXIT_FAILURE` otherwise.

```
void libxs_gemm_release(libxs_gemm_config_t* config);
```

Release resources acquired by `libxs_gemm_dispatch` (e.g., MKL jitter). Safe to call even if dispatch was not used or returned zero.

```
void libxs_gemm_release_registry(libxs_registry_t* registry);
```

Release all GEMM configs cached in a registry (e.g., MKL jitter), then destroy the registry itself. Safe to call with NULL.

```
void libxs_gemm_batch(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config);
```

Process a batch of GEMMs given arrays of pointers to matrices. Shape, alpha, beta, datatype, and transpose info come from `config->shape`.

```
void libxs_gemm_batch_task(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Per-thread form of `libxs_gemm_batch`.

```
void libxs_gemm_index(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config);
```

Process a batch of GEMMs given index arrays into contiguous buffers. Each `stride_a[i]`, `stride_b[i]`, `stride_c[i]` is an element-offset from the respective base pointer. `index_base` selects the indexing convention: 0 for zero-based (C), 1 for one-based (Fortran). `index_stride` is the Byte-stride used to walk the stride arrays (e.g., `sizeof(int)` for packed int arrays). `index_stride=0` selects constant-stride mode: each stride array points to a single element, and batch `i` uses offset `stride[0]*i` (equivalent to the strided batch pattern). Shape, alpha, beta, datatype, and transpose info come from `config->shape`.

```
void libxs_gemm_index_task(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Per-thread form of `libxs_gemm_index`.

Histogram

Header: `libxs_hist.h`

Thread-safe histogram that accumulates values and provides running statistics. Values are bucketed by the first element of each inserted tuple; additional per-entry values track user-defined statistics (e.g., timing data).

Types

```
typedef struct libxs_hist_t libxs_hist_t;  /* opaque */
```

```
typedef void (*libxs_hist_update_t)(double* dst, const double* src);
```

Per-value accumulation callback. Two built-in update functions are provided: `libxs_hist_update_avg` (sliding average) and `libxs_hist_update_add` (additive accumulation).

Functions

```
libxs_hist_t* libxs_hist_create(int nbuckets, int nvals,
  const libxs_hist_update_t update[]);
```

Create a histogram with `nbuckets` resolution. `nvals` is the number of values per entry (the first value determines the bucket). The `update` array (length `nvals`) specifies the accumulation function for each value slot; NULL entries default to `libxs_hist_update_avg`. Returns NULL on failure.

```
void libxs_hist_destroy(libxs_hist_t* hist);
```

Destroy a histogram. Accepts NULL.

```
void libxs_hist_push(libxs_lock_t* lock,
  libxs_hist_t* hist, const double vals[]);
```

Insert one sample. `vals[0]` selects the bucket; remaining entries are accumulated according to the update functions. The `lock` parameter can be NULL (internal locking is used).

```
void libxs_hist_get(libxs_lock_t* lock,
  const libxs_hist_t* hist,
  const int** buckets, int* nbuckets,
  double range[2], const double** vals, int* nvals);
```

Query statistics. Output pointers can be NULL to skip fields.

```
void libxs_hist_get_percentile(libxs_lock_t* lock,
  const libxs_hist_t* hist, double percentile,
  double vals[]);
```

Query interpolated values at the given percentile. `percentile` is in the range `[0..1]` (clamped). `vals[]` is filled with interpolated results: `vals[0]` is the interpolated primary value (position along the range), and subsequent entries are linearly interpolated between adjacent buckets. Commits queued items if pending.

```
void libxs_hist_get_median(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[]);
```

Convenience wrapper equivalent to `libxs_hist_get_percentile(lock, hist, 0.5, vals)`.

```
void libxs_hist_print(FILE* ostream,
  const libxs_hist_t* hist, const int prec[],
  const char fmt[], ...);
```

Render the histogram to a stream. `prec` controls decimal precision per value (can be NULL); a negative `prec[k]` omits that value from output, and a negative `prec[0]` skips the bucket line entirely. `fmt` is a printf-style format string for the title line, followed by optional variadic arguments. NULL `ostream` or `fmt` is accepted.

```
void libxs_hist_update_avg(double* dst, const double* src);
void libxs_hist_update_add(double* dst, const double* src);
```

Built-in update functions: `libxs_hist_update_avg` computes a sliding average (arithmetic mean), `libxs_hist_update_add` sums values.

Memory Allocation

Header: `libxs_malloc.h`

Pool-based memory allocator designed for steady-state performance: after an initial warm-up phase, allocations are served from a recycled pool without system calls.

Types

```
typedef struct libxs_malloc_info_t {
  size_t size;  /* allocated size in bytes */
} libxs_malloc_info_t;

typedef struct libxs_malloc_pool_info_t {
  size_t used;      /* memory currently in use (sum of requested sizes) */
  size_t size;      /* total allocated memory (sum of actual chunk sizes) */
  size_t peak;      /* peak memory consumption */
  size_t nactive;   /* pending (not yet freed) allocations */
  size_t nmallocs;  /* total allocation count */
} libxs_malloc_pool_info_t;

typedef struct libxs_malloc_pool_t libxs_malloc_pool_t; /* opaque */

typedef void* (*libxs_malloc_fn)(size_t size);
typedef void  (*libxs_free_fn)(void* pointer);

typedef void* (*libxs_malloc_xfn)(size_t size, const void* extra);
typedef void  (*libxs_free_xfn)(void* pointer, const void* extra);
```

Pool Management

```
libxs_malloc_pool_t* libxs_malloc_pool(libxs_malloc_fn malloc_fn, libxs_free_fn free_fn);
```

Create a memory pool. If `malloc_fn` and `free_fn` are both NULL, the standard `malloc`/`free` are used. Both must be NULL or both must be non-NULL.

```
libxs_malloc_pool_t* libxs_malloc_xpool(libxs_malloc_xfn malloc_fn, libxs_free_xfn free_fn,
  int max_nthreads);
```

Create a memory pool with extended allocator functions that receive a per-thread extra argument (see `libxs_malloc_arg`). The `max_nthreads` parameter determines the size of the internal per-thread argument table (indexed by `libxs_tid`). Both function pointers must be non-NULL and `max_nthreads` must be positive; otherwise NULL is returned.

```
void libxs_malloc_arg(libxs_malloc_pool_t* pool, const void* extra);
```

Set the per-thread extra argument for an extended pool (created by `libxs_malloc_xpool`). The pointer is stored at the calling thread's slot (`libxs_tid() % max_nthreads`) and passed to the registered `malloc_xfn`/`free_xfn` on subsequent allocations and frees from this thread. No-op for standard pools. Access is lock-free since each thread writes only its own slot.

```
void libxs_free_pool(libxs_malloc_pool_t* pool);
```

Destroy the pool and release all associated memory. Accepts NULL.

```
int libxs_malloc_pool_info(const libxs_malloc_pool_t* pool, libxs_malloc_pool_info_t* info);
```

Query aggregate pool statistics.

General Allocation

```
void* libxs_malloc(libxs_malloc_pool_t* pool, size_t size, int alignment);
```

Allocate `size` bytes from the given `pool`. `LIBXS_MALLOC_AUTO` uses automatic alignment with inline metadata. `LIBXS_MALLOC_NATIVE` preserves the allocator's native pointer (out-of-band metadata via registry). Values greater than 1 are interpreted as explicit alignment in Bytes (inline metadata). Returns NULL on failure or if `pool` is NULL.

```
void libxs_free(void* pointer);
```

Return memory to its originating pool (derived internally). Accepts NULL.

```
int libxs_malloc_info(const void* pointer, libxs_malloc_info_t* info);
```

Query the size of an allocation made with `libxs_malloc`. The pool is derived internally.

Fixed-Size Pool

A lightweight fixed-size pool for scenarios where the element size is known at initialization time.

```
void libxs_pmalloc_init(size_t size, size_t* num,
  void* pool[], void* storage);
```

Partition `storage` into `*num` chunks of `size` bytes and register them in `pool`.

```
void* libxs_pmalloc(void* pool[], size_t* num);
void* libxs_pmalloc_lock(void* pool[], size_t* num,
  libxs_lock_t* lock);
```

Pop one chunk from the pool. The `_lock` variant uses a caller-provided lock; the plain variant uses an internal lock.

```
void libxs_pfree(void* pointer, void* pool[], size_t* num);
void libxs_pfree_lock(void* pointer, void* pool[],
  size_t* num, libxs_lock_t* lock);
```

Push a chunk back into the pool.

Math Utilities

Header: `libxs_math.h`

Matrix Difference

```
typedef struct libxs_matdiff_t {
  double norm1_abs, norm1_rel;  /* one-norm */
  double normi_abs, normi_rel;  /* infinity-norm */
  double normf_rel;             /* Frobenius-norm (relative) */
  double linf_abs, linf_rel;    /* max difference (abs/rel at same element) */
  double l2_abs, l2_rel, rsq;   /* L2-norm and R-squared */
  double l1_ref, min_ref, max_ref, avg_ref, var_ref;  /* reference stats */
  double l1_tst, min_tst, max_tst, avg_tst, var_tst;  /* test stats */
  double diag_min_ref, diag_max_ref;   /* diagonal min/max (reference) */
  double diag_min_tst, diag_max_tst;   /* diagonal min/max (test) */
  double v_ref, v_tst;          /* values at max-diff location */
  int m, n, i, r;               /* location and reduction count */
} libxs_matdiff_t;
```

The fields linf_abs, linf_rel, and v_ref/v_tst always refer to the same element. For complex types, element values and statistics use the modulus; differences use the complex absolute error.

```
int libxs_matdiff(libxs_matdiff_t* info,
  libxs_data_t datatype, int m, int n,
  const void* ref, const void* tst,
  const int* ldref, const int* ldtst);
```

Compute scalar differences between two matrices. Supports all real and integer `libxs_data_t` types, plus `LIBXS_DATATYPE_C64` and `LIBXS_DATATYPE_C32` (interleaved complex; dimensions refer to complex elements).

```
double libxs_matdiff_epsilon(const libxs_matdiff_t* input);
```

Combined error margin from absolute and relative norms. Optionally logs to file via `LIBXS_MATDIFF` env var.

```
int libxs_matdiff_combine(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
```

Combine two single-matrix infos (`ref=NULL`) into a meta-diff. Per-side statistics are exact; `linf_abs` is the mean shift, `l2_abs` a statistical bound.

```
void libxs_matdiff_reduce(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
void libxs_matdiff_clear(libxs_matdiff_t* info);
```

Worst-case reduction of matdiff results. Initialize with `libxs_matdiff_clear`.

```
double libxs_matdiff_posdef(const libxs_matdiff_t* info);  /* inline */
```

Returns the smallest test-side diagonal element (positive = necessary condition for positive definiteness met). Zero if no diagonal data.

Number Theory

```
size_t libxs_gcd(size_t a, size_t b);
size_t libxs_lcm(size_t a, size_t b);
```

GCD (returns 1 for gcd(0,0)) and LCM.

```
int libxs_primes_u32(unsigned int num,
  unsigned int num_factors_n32[], int num_factors_max);
```

Prime factors of `num` in ascending order. Returns factor count.

```
size_t libxs_coprime(size_t n, size_t minco);
size_t libxs_coprime2(size_t n);
```

Co-prime of `n` not exceeding `minco` (or sqrt(n)).

```
unsigned int libxs_remainder(unsigned int a, unsigned int b,
  const unsigned int* limit, const unsigned int* remainder);
```

Smallest multiple of `b` minimizing remainder mod `a`.

```
unsigned int libxs_product_limit(unsigned int product,
  unsigned int limit, int is_lower);
```

Sub-product of prime factors within `limit` (0/1-Knapsack).

Scalar Utilities

```
double libxs_kahan_sum(double value, double* accumulator,
  double* compensation);
```

Kahan compensated summation.

```
unsigned int libxs_isqrt_u64(unsigned long long x);
unsigned int libxs_isqrt_u32(unsigned int x);
unsigned int libxs_isqrt2_u32(unsigned int x);
```

Integer square root. The isqrt2 variant returns a factor of x.

```
double libxs_pow2(int n);
```

2ˆn via IEEE-754 exponent. Valid for n in [-1022, 1023]. Returns 0.0 for underflow (subnormals flushed to zero) and +Inf for overflow.

Modular Arithmetic

```
unsigned int libxs_mod_inverse_u32(unsigned int a, unsigned int m);
```

Modular inverse (extended Euclidean). Requires gcd(a, m) = 1.

```
unsigned int libxs_barrett_rcp(unsigned int p);
unsigned int libxs_barrett_pow18(unsigned int p);
unsigned int libxs_barrett_pow36(unsigned int p);
```

Barrett reduction constants for modulus `p`.

```
unsigned int libxs_mod_u32(uint32_t x, unsigned int p,
  unsigned int rcp);                          /* inline */
unsigned int libxs_mod_u64(uint64_t x, unsigned int p,
  unsigned int rcp, unsigned int pow18,
  unsigned int pow36);                        /* inline */
```

Fast modular reduction via Barrett. The 64-bit variant uses a radix-2ˆ18 split.

BF16 Conversion

```
typedef uint16_t libxs_bf16_t;
libxs_bf16_t libxs_round_bf16(double x);   /* inline */
double libxs_bf16_to_f64(libxs_bf16_t v);  /* inline */
```

Round to BF16 (round-to-nearest-even) and expand to double. Uses native `__bf16` when available (`LIBXS_BF16`), otherwise portable bit manipulation.

Memory and Hashing Utilities

Header: `libxs_mem.h`

Byte-level memory operations (macros), comparison, hashing, string search, value formatting, and data-shuffling primitives.

Memory Macros

```
LIBXS_MEMSET(DST, SRC, SIZE)
```

Set `SIZE` bytes at `DST` to the value `SRC`, unrolled at compile-time.

```
LIBXS_MEMZERO(DST)
```

Zero all bytes of `*DST` (size derived from `sizeof(*DST)`).

```
LIBXS_MEMCPY(DST, SRC, SIZE)
```

Copy `SIZE` bytes from `SRC` to `DST`, unrolled at compile time.

```
LIBXS_ASSIGN(DST, SRC)
```

Copy `sizeof(*SRC)` bytes from `SRC` to `DST`. Asserts `sizeof(*SRC) <= sizeof(*DST)`.

```
LIBXS_MEMSWP(PTR_A, PTR_B, SIZE)
```

Swap `SIZE` bytes between `PTR_A` and `PTR_B`. Asserts `PTR_A != PTR_B`.

```
LIBXS_VALUE_ASSIGN(DST, SRC)
LIBXS_VALUE_SWAP(A, B)
```

Operate on L-values directly (address-of is taken internally). `VALUE_ASSIGN` can cast away const qualifiers. `VALUE_SWAP` asserts equal size.

Offset and Alignment

```
size_t libxs_offset(size_t ndims, const size_t offset[],
  const size_t shape[], size_t* size);
```

Compute the linear offset of an n-dimensional coordinate within a shape. Optionally returns the total linear size of the shape.

```
int libxs_aligned(const void* ptr, const size_t* inc,
  int* alignment);
```

Check whether `ptr` is SIMD-aligned (optionally considering the next access at `ptr + *inc`). Optionally writes the pointer alignment in bytes to `*alignment`.

Comparison

```
unsigned char libxs_diff(const void* a, const void* b,
  unsigned char size);
```

Compare two short buffers. Returns zero when equal, non-zero otherwise.

```
unsigned int libxs_diff_n(const void* a, const void* bn,
  unsigned char elemsize, unsigned char stride,
  unsigned int hint, unsigned int count);
```

Compare `a` against `count` elements in `bn`. Returns the first index that matches `a`, or `count` if none match. `hint` supplies the initial search position.

```
int libxs_memcmp(const void* a, const void* b, size_t size);
```

Boolean variant of the C `memcmp`. Returns zero when equal.

Hashing

```
unsigned int libxs_hash(const void* data, unsigned int size,
  unsigned int seed);
unsigned int libxs_hash8(unsigned int data);
unsigned int libxs_hash16(unsigned int data);
unsigned int libxs_hash32(unsigned long long data);
```

Produce a 32-bit hash from a buffer (with seed) or from an 8/16/32-bit key directly. NULL buffer is accepted (returns seed).

```
unsigned long long libxs_hash_string(const char string[]);
```

64-bit hash of a character string. NULL-string is accepted.

String Search

```
const char* libxs_stristrn(const char a[], const char b[],
  size_t maxlen);
const char* libxs_stristr(const char a[], const char b[]);
```

Case-insensitive substring search. `stristrn` limits the search to `maxlen` characters. Returns the pointer to the first match, or NULL.

```
int libxs_strimatch(const char a[], const char b[],
  const char delims[], int* count);
```

Word-level fuzzy matching. Counts how many words in `a` (or `b`) appear in the other string (case-insensitive, symmetric). Optional `delims` define word separators; optional `count` receives the total word count.

Value Formatting

```
size_t libxs_format_value(char buffer[], int buffer_size,
  size_t nbytes, const char scale[], const char* unit, int base);
```

Format a scalar value with SI-style scaling. Example: `libxs_format_value(buf, sizeof(buf), nbytes, "KMGT", "B", 10)` produces a human-readable byte count. Returns the value in the selected unit.

Matrix Copy and Transposition

```
void libxs_matcopy(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_matcopy_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Copy an m-by-n matrix from `in` to `out` with leading dimensions `ldi` and `ldo`. If `in` is NULL the destination is zeroed. The `_task` variant distributes work across `ntasks` threads (caller passes its `tid`).

```
void libxs_otrans(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_otrans_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Out-of-place matrix transposition. The `_task` variant distributes work across `ntasks` threads.

```
void libxs_itrans(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo, void* scratch);
void libxs_itrans_task(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo, void* scratch,
  int tid, int ntasks);
```

In-place matrix transposition (square or via scratch buffer). `scratch` can be NULL (auto-allocated).

```
void libxs_itrans_batch(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo,
  int index_base, int index_stride,
  const int stride[], int batchsize,
  int tid, int ntasks);
```

Batch of in-place matrix transpositions (per-thread form).

Shuffling

```
int libxs_shuffle(void* inout, size_t elemsize, size_t count,
  const size_t* shuffle, const size_t* nrepeat);
```

In-place shuffle of `count` elements. The `shuffle` stride must be co-prime to `count` (NULL selects `libxs_coprime2(count)`). `nrepeat` defaults to 1.

```
int libxs_shuffle2(void* dst, const void* src, size_t elemsize,
  size_t count, const size_t* shuffle, const size_t* nrepeat);
```

Out-of-place shuffle from `src` to `dst`. If `*nrepeat` is zero an ordinary copy is performed.

```
size_t libxs_unshuffle(size_t count, const size_t* shuffle);
```

Return the number of `libxs_shuffle2` applications needed to restore the original order.

MetaImage I/O

Header: `libxs_mhd.h`

Read and write image data in the MetaImage (MHD/MHA) format. Files are compatible with ITK-SNAP, ParaView, and similar tools.

Types

```
typedef enum libxs_mhd_element_conversion_hint_t {
  LIBXS_MHD_ELEMENT_CONVERSION_DEFAULT,
  LIBXS_MHD_ELEMENT_CONVERSION_MODULUS
} libxs_mhd_element_conversion_hint_t;
```

Hint controlling element conversion behaviour (default clamping vs. modulus mapping).

```
typedef struct libxs_mhd_element_handler_info_t {
  libxs_data_t type;
  libxs_mhd_element_conversion_hint_t hint;
} libxs_mhd_element_handler_info_t;
```

Destination type and conversion hint, passed to built-in or custom element handlers.

```
typedef int (*libxs_mhd_element_handler_t)(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max);
```

Per-element callback for reading or converting image data. The value range (`src_min`, `src_max`) is available for scaling.

```
typedef struct libxs_mhd_info_t {
  size_t ndims;         /* dimensionality */
  size_t ncomponents;   /* interleaved image channels */
  libxs_data_t type;  /* pixel element type */
  size_t header_size;   /* header size in bytes (LOCAL data) */
} libxs_mhd_info_t;
```

Image descriptor populated by `libxs_mhd_read_header` and consumed by read/write.

```
typedef struct libxs_mhd_write_info_t {
  const libxs_mhd_element_handler_info_t* handler_info;
  libxs_mhd_element_handler_t handler;
  const char* extension_header;
  const void* extension;
  size_t extension_size;
} libxs_mhd_write_info_t;
```

Optional write parameters (conversion, custom handler, extension data). NULL is accepted for all-defaults.

Element Helpers

```
int libxs_mhd_element_conversion(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max);
```

Built-in element conversion. Scales values when `src_min` and `src_max` are non-NULL; otherwise clamps to the destination type.

```
int libxs_mhd_element_comparison(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max);
```

Check an in-memory buffer against file content (element-wise comparison with optional type conversion).

Type Queries

```
const char* libxs_mhd_typename(libxs_data_t type,
  const char** ctypename);
```

Return the MHD element-type name (e.g. `"MET_FLOAT"`) for the given `libxs_data_t`. Optionally writes the C type name. Returns NULL for unknown types.

```
libxs_data_t libxs_mhd_typeinfo(const char elemname[]);
```

Return the `libxs_data_t` for a given MHD element-type string.

Read

```
int libxs_mhd_read_header(const char header_filename[],
  size_t filename_max_length, char filename[],
  libxs_mhd_info_t* info, size_t size[],
  char extension[], size_t* extension_size);
```

Parse an MHD header file. `info->ndims` is an in/out parameter (maximum on input, actual on output). `filename` receives the data-file path (may differ from the header when data is stored separately). Extension data is optional.

```
int libxs_mhd_read(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], const libxs_mhd_info_t* info,
  void* data,
  const libxs_mhd_element_handler_info_t* handler_info,
  libxs_mhd_element_handler_t handler);
```

Read image data into `data`. Optional `handler_info` triggers type conversion; optional `handler` supplies a custom per-element callback (can also serve as a comparison function without allocating a buffer).

Write

```
int libxs_mhd_write(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], libxs_mhd_info_t* info,
  const void* data,
  const libxs_mhd_write_info_t* write_info);
```

Write image data in the extended MHD format. `info->header_size` is updated on output. Pass NULL for `write_info` when no conversion or extension is needed.

Visit LIBXSTREAM

Start Presentation

Registry

Header: `libxs_reg.h`

Thread-safe key-value store backed by a hash table with per-thread caching.

Configuration Macros

| Macro | Default | Description | |:|:|:| | `LIBXS_REGKEY_MAXSIZE` | 64 | Maximum key size in bytes | | `LIBXS_REGISTRY_NBUCKETS` | 64 | Initial number of hash-table buckets (must be a power of two) | | `LIBXS_REGCACHE_NENTRIES` | 16 | Thread-local cache entries per thread (power of two; 0 disables caching) |

Types

```
typedef struct libxs_registry_t libxs_registry_t;   /* opaque */

typedef struct libxs_registry_info_t {
  size_t capacity, size, nbytes;
} libxs_registry_info_t;
```

Lifetime

```
libxs_registry_t* libxs_registry_create(void);
```

Create a new registry. Returns `NULL` in case of an error.

```
void libxs_registry_destroy(libxs_registry_t* registry);
```

Destroy a registry and release all entries.

```
libxs_lock_t* libxs_registry_lock(libxs_registry_t* registry);
```

Return a pointer to the registry's internal lock. The returned lock can be passed as the `lock` argument to other registry functions (e.g., to hold the lock across multiple operations).

Iteration

```
void* libxs_registry_begin(libxs_registry_t* registry,
  const void** key, size_t* cursor);
void* libxs_registry_next(libxs_registry_t* registry,
  const void** key, size_t* cursor);
```

Enumerate entries. Initialize `*cursor` to 0 before the first `begin` call. Each call returns the value pointer and writes the key pointer to `*key`, or returns NULL when iteration is complete.

Access

```
void* libxs_registry_set(libxs_registry_t* registry,
  const void* key, size_t key_size,
  const void* value_init, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Insert or update a key-value pair. The key must be binary-reproducible (zero-initialize padding). `value_init` may be NULL to defer initialization. Re-registering an existing key reallocates if the new value is larger. When `lock` is NULL the registry's internal lock is used. Returns a pointer to the stored value.

```
void* libxs_registry_get(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Look up a value by key. Returns NULL if not found.

```
int libxs_registry_get_copy(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Thread-safe query: copies up to `value_size` bytes of the stored value into `value_out` under the lock. Unlike `libxs_registry_get`, the caller never receives a raw pointer into the registry's internal storage, making this safe under concurrent modifications. Returns non-zero if the key was found, zero otherwise.

```
int libxs_registry_has(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Check whether a key exists. Non-zero if found.

```
size_t libxs_registry_value_size(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Query the stored value size in bytes. Returns 0 if not found.

```
void libxs_registry_remove(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Remove a key-value pair and release its memory.

```
int libxs_registry_extract(libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Atomically retrieve and remove a key-value pair. Copies up to `value_size` bytes of the stored value into `value_out` (may be NULL to discard the value), then removes the entry and releases its memory. The lookup, copy, and removal are performed under a single lock hold, eliminating the race that exists when calling `libxs_registry_get` followed by `libxs_registry_remove` separately. Returns non-zero if the key was found, zero otherwise.

Status

```
int libxs_registry_info(libxs_registry_t* registry,
  libxs_registry_info_t* info);
```

Query registry capacity, number of entries, and total bytes stored.

Random Number Generator

Header: `libxs_rng.h`

Thread-safe pseudo-random number generation (SplitMix64 with per-thread TLS state).

Matrix Initialization Macros

```
LIBXS_MATRNG(INT_TYPE, REAL_TYPE, SEED, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_SEQ(INT_TYPE, REAL_TYPE, SEED, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_OMP(INT_TYPE, REAL_TYPE, SEED, DST, NROWS, NCOLS, LD, SCALE)
```

Fill a column-major matrix (`NROWS` x `NCOLS`, leading dimension `LD`) with deterministic values. When `SEED != 0` the fill is linear; when `SEED == 0` a shuffle-based initialization normalizes values into [-SCALE, +SCALE]. The `_OMP` variant parallelizes with OpenMP; `_SEQ` is serial.

Functions

```
void libxs_rng_set_seed(unsigned int seed);
```

Set the seed of the calling thread's PRNG state. Each thread maintains independent state via TLS. Unseeded threads start with a deterministic default (seed = 1).

```
unsigned int libxs_rng_u32(unsigned int n);
```

Return a pseudo-random value in [0, n) with uniform distribution (Lemire's nearly-divisionless method). Thread-safe.

```
double libxs_rng_f64(void);
```

Return a double-precision value in [0, 1) with full 53-bit mantissa resolution. Thread-safe.

```
void libxs_rng_seq(void* data, size_t nbytes);
```

Fill a buffer with pseudo-random bytes. Thread-safe.

Synchronization

Header: `libxs_sync.h`

Thread-local storage, atomic operations, lock abstractions (spin/mutex/rwlock/atomic), file locking, and stdio synchronization.

Thread-Local Storage

`LIBXS_TLS`

Storage-class qualifier for thread-local variables. Maps to `__thread`, `__declspec(thread)`, or `thread_local` depending on the compiler. Defined as empty when TLS is unavailable or disabled (`LIBXS_NO_TLS`).

Atomic Operations

The header provides a suite of macros for atomic loads, stores, compare-and-swap, and arithmetic. The implementation selects among GCC builtins (`__atomic_*`), legacy GCC sync builtins, Windows Interlocked intrinsics, or no-op fallbacks depending on the compiler and `LIBXS_SYNC` setting.

| Macro | Description | |:|:| | `LIBXS_ATOMIC_LOAD(SRC, KIND)` | Atomic load | | `LIBXS_ATOMIC_STORE(DST, VALUE, KIND)` | Atomic store | | `LIBXS_ATOMIC_STORE_ZERO(DST, KIND)` | Atomic store of zero | | `LIBXS_ATOMIC_CMPSWP(DST, OLDVAL, NEWVAL, KIND)` | Compare-and-swap | | `LIBXS_ATOMIC_FETCH_OR(DST, VALUE, KIND)` | Fetch-and-or | | `LIBXS_ATOMIC_FETCH_ADD(DST, VALUE, KIND)` | Fetch-and-add | | `LIBXS_ATOMIC_FETCH_SUB(DST, VALUE, KIND)` | Fetch-and-subtract | | `LIBXS_ATOMIC_ADD_FETCH(DST, VALUE, KIND)` | Add-and-fetch | | `LIBXS_ATOMIC_SUB_FETCH(DST, VALUE, KIND)` | Subtract-and-fetch | | `LIBXS_ATOMIC_TRYLOCK(DST, KIND)` | Try-lock (returns acquired state) | | `LIBXS_ATOMIC_ACQUIRE(DST, NPAUSE, KIND)` | Spin-acquire with backoff | | `LIBXS_ATOMIC_RELEASE(DST, KIND)` | Release (store zero) | | `LIBXS_ATOMIC_SYNC(KIND)` | Full memory fence |

The `KIND` parameter selects the memory order but is ignored on most backends (sequential consistency is always used).

Lock Abstraction

```
LIBXS_LOCK_TYPE(KIND)
LIBXS_LOCK_INIT(KIND, LOCK, ATTR)
LIBXS_LOCK_DESTROY(KIND, LOCK)
LIBXS_LOCK_ACQUIRE(KIND, LOCK)
LIBXS_LOCK_TRYLOCK(KIND, LOCK)
LIBXS_LOCK_RELEASE(KIND, LOCK)
```

Generic lock interface parameterized by `KIND`:

| KIND | Backend | |:|:| | `LIBXS_LOCK_SPINLOCK` | Atomic spin-lock | | `LIBXS_LOCK_MUTEX` | Pthreads / Windows CRITICAL_SECTION | | `LIBXS_LOCK_RWLOCK` | Pthreads reader-writer lock | | `LIBXS_LOCK_ATOMIC` | Lightweight atomic lock |

The default lock kind used by the library is `LIBXS_LOCK` (resolves to one of the above based on build configuration).

```
typedef LIBXS_LOCK_TYPE(LIBXS_LOCK) libxs_lock_t;
```

General-purpose lock type. Instances of `libxs_lock_t` are used by the registry and other library components; users may also create their own.

Reader-writer variants are available for `LIBXS_LOCK_RWLOCK`:

```
LIBXS_LOCK_ACQREAD(KIND, LOCK)
LIBXS_LOCK_RELREAD(KIND, LOCK)
LIBXS_LOCK_TRYREAD(KIND, LOCK)
```

File Locking

```
LIBXS_FLOCK(FILE)
LIBXS_FUNLOCK(FILE)
```

Per-file locking for thread-safe I/O. Maps to `flockfile`/`funlockfile` on POSIX, `_lock_file`/`_unlock_file` on Windows, or no-ops when synchronization is disabled.

Functions

```
unsigned int libxs_nranks(void);
```

Return the number of MPI ranks.

```
unsigned int libxs_nrank(void);
```

Return the MPI rank of the calling process.

```
unsigned int libxs_rid(void);
```

Return a rank ID of the calling process.

```
unsigned int libxs_pid(void);
```

Return the process ID of the calling process.

```
unsigned int libxs_tid(void);
```

Return a zero-based, consecutive thread ID for the calling thread. TID = 0 does not necessarily correspond to the main thread.

```
void libxs_stdio_acquire(void);
void libxs_stdio_release(void);
```

Acquire/release a global lock around console output. The macros `LIBXS_STDIO_ACQUIRE()` and `LIBXS_STDIO_RELEASE()` expand to these calls.

Timer

Header: `libxs_timer.h`

High-resolution timing via a calibrated TSC when available, with an OS real-time clock fallback.

Types

```
typedef unsigned long long libxs_timer_tick_t;
```

Integer type representing a single timer sample.

```
typedef struct libxs_timer_info_t {
  int tsc;   /* non-zero if a calibrated TSC is used */
} libxs_timer_info_t;
```

Functions

```
int libxs_timer_info(libxs_timer_info_t* info);
```

Query timer properties. `info->tsc` is non-zero when RDTSC (x86), CNTVCT_EL0 (AArch64), or MFTB (PPC64) is available. Triggers lazy library initialization if necessary.

```
libxs_timer_tick_t libxs_timer_tick(void);
```

Sample the current tick of the monotonic timer. The tick unit is platform-specific; convert with `libxs_timer_duration` or `libxs_timer_ncycles`.

```
libxs_timer_tick_t libxs_timer_ncycles(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);   /* inline */
```

Unsigned difference `tick1 - tick0`, handling wrap-around safely. Implemented via `LIBXS_DELTA`.

```
double libxs_timer_duration(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);
```

Convert a pair of ticks to elapsed wall-clock time in seconds.

Low-Level Utilities and SIMD

Header: `libxs_utils.h`

Target-attribute machinery, ISA feature gates, intrinsic fix-ups, bit-scan operations, and AVX-512 inline helpers.

Target Attributes

```
LIBXS_INTRINSICS(TARGET)
```

Expands to a `__attribute__((target(...)))` clause for the given ISA level. Used to compile individual functions for a higher ISA than the baseline without changing compiler flags. The macro is a no-op when the baseline already covers the requested level or when the compiler does not support multi-versioning.

```
LIBXS_ATTRIBUTE_TARGET(TARGET)
```

Lower-level macro that resolves `TARGET` (an integer ISA constant such as `LIBXS_X86_AVX512`) to the corresponding target string, e.g. `target("avx2,fma,avx512f,...")`.

ISA Feature Gates

Preprocessor symbols defined when the compiler can generate code for a given ISA:

Macro	ISA level
`LIBXS_INTRINSICS_X86`	x86 baseline (SSE2)
`LIBXS_INTRINSICS_SSE3`	SSE3
`LIBXS_INTRINSICS_SSE42`	SSE4.2
`LIBXS_INTRINSICS_AVX`	AVX
`LIBXS_INTRINSICS_AVX2`	AVX2 + FMA
`LIBXS_INTRINSICS_AVX512`	AVX-512 (F + CD + DQ + BW + VL + VNNI)

`LIBXS_STATIC_TARGET_ARCH` records the highest ISA provided by the compiler flags. `LIBXS_MAX_STATIC_TARGET_ARCH` records the highest ISA reachable via target attributes. Target attributes (`LIBXS_ATTRIBUTE_TARGET_*`) are defined up to `LIBXS_X86_AVX10_512` depending on compiler version.

Intrinsic Fix-ups

Portability wrappers for intrinsics whose signatures differ across compilers:

| Macro | Purpose | |:|:| | LIBXS_INTRINSICS_LOADU_SI128 | _mm_loadu_si128 | | LIBXS_INTRINSICS_MM512_LOAD_PS/PD | Unaligned 512-bit float/double load | | LIBXS_INTRINSICS_MM512_STREAM_* | Non-temporal 512-bit stores | | LIBXS_INTRINSICS_MM256_STORE_EPI32 | 256-bit integer store | | LIBXS_INTRINSICS_MM512_SET_EPI16 | Portable 512-bit set from 32 int16 values | | LIBXS_INTRINSICS_MM512_UNDEFINED_* | Undefined-value initializers (zero in debug) | | LIBXS_INTRINSICS_MM512_EXTRACTI64X4_EPI64 | 256-bit extract from 512-bit | | LIBXS_INTRINSICS_MM512_ABS_PS | Absolute value of packed floats | | LIBXS_INTRINSICS_MM512_MASK_I32GATHER_EPI32 | Masked gather | | LIBXS_INTRINSICS_MM512_STORE/LOAD_MASK* | Mask register I/O (8/16/32/64-bit) | | LIBXS_INTRINSICS_MM512_CVTU32_MASK* | Convert uint32 to mask register |

Bit-Scan Operations

```
LIBXS_INTRINSICS_BITSCANFWD32(N)   /* count trailing zeros, 32-bit */
LIBXS_INTRINSICS_BITSCANFWD64(N)   /* count trailing zeros, 64-bit */
LIBXS_INTRINSICS_BITSCANBWD32(N)   /* bit index of highest set bit, 32-bit */
LIBXS_INTRINSICS_BITSCANBWD64(N)   /* bit index of highest set bit, 64-bit */
```

Use GCC `__builtin_ctz`/`__builtin_clz` when available, Windows `_BitScanForward`/`_BitScanReverse` intrinsics on MSVC, or portable software fallbacks (`*_SW` variants). All return 0 when `N` is 0.

Derived Macros

```
LIBXS_NBITS(N)      /* minimum bits to represent N */
LIBXS_ISQRT2(N)     /* fast power-of-two approximation of sqrt(N) */

unsigned int LIBXS_ILOG2(unsigned long long n);   /* ceil(log2(n)) */
```

`LIBXS_ILOG2` is a function (not a macro) and handles the n = 0 case.

MXCSR (FP Environment)

Macros for preserving and restoring the x86 MXCSR register (floating-point control/status). On non-x86 platforms these are no-ops.

| Macro | Description | |:|:| | LIBXS_MXCSR_FTZ | Flush-to-zero flag (0x8000 on x86, 0 otherwise) | | LIBXS_MXCSR_DAZ | Denormals-are-zero flag (0x0040 on x86, 0 otherwise) | | LIBXS_MXCSR_GET() | Read the MXCSR register (_mm_getcsr() on x86, 0 otherwise) | | LIBXS_MXCSR_SET(VALUE) | Write the MXCSR register (_mm_setcsr() on x86, no-op otherwise) |

AVX-512 Inline Functions

Available only when `LIBXS_INTRINSICS_AVX512` is defined.

```
__m512i libxs_mulhi_epu32(__m512i a, __m512i b);   /* inline */
```

Unsigned 32-bit high-multiply for 16 lanes: floor(a * b / 2^32). Emulates the missing `_mm512_mulhi_epu32` via even/odd `_mm512_mul_epu32`.

```
__m512i libxs_mod_u32x16(__m512i x,
  unsigned int p, unsigned int rcp);               /* inline */
```

Vectorized Barrett reduction: x mod p for 16 uint32 lanes. `rcp` is the Barrett reciprocal from `libxs_barrett_rcp()`. This is the SIMD counterpart of the scalar `libxs_mod_u32`.

Appendix

Overview

LIBXS's collection of scripts consists of Python and Bash scripts falling into two categories:

- Configuration scripts
- Development tools

Scripts related to configuring LIBXS are distributed with source code archives. Development tools mostly for software development purpose and are (indirectly) used by contributors, but some scripts are distributed by source code archives as well. The latter are mostly related to running tests (indirectly used by upstream maintainers, e.g., of Linux distributions).

Development

Parallel Execution

The script `tool_pexec.sh` can execute commands read from standard input (see `-h` or `--help`). The execution may be concurrent on a per-command basis. The level of parallelism is determined automatically but can be adjusted (oversubscription, nested parallelism). Separate logfiles can be written for every executed command (`-o /path/to/basename.ext` used as template for individual logfiles like /path/to/*basename*-case_xyz.*ext*). File I/O can become a bottleneck on distributed filesystems (e.g., NFS), or generally hinders nested parallelism (`-o /dev/null -k`).

Every line of standard input denotes a separate command:

```
seq 100 | xargs -I{} echo "echo \"{}\"" \
        | tool_pexec.sh
```

The script considers an allow-list which permits certain error codes. Allow-lists can be automatically generated (`-u`). Most if not all settings can be determined by environment variables as well (prefix `PEXEC_`), e.g., `export PEXEC_LG=/path/to/basename.ext` allows to omit `-o` on the command line and to always generate logfiles (see earlier explanation).

Performance Report The script `tool_report.py` collects performance results given in two possible formats: (1) native "telegram" format, and (2) JSON format. The script aims to avoid encoding domain knowledge. In fact, the collected information is not necessarily performance data but a time series in general. Usually, the script is not executed directly but launched using a wrapper supplying the authorization token and further adapting to the execution environment (setup):

```
#!/usr/bin/env bash

## authorization token
TOKEN=0123456789abcdef0123456789abcdef01234567

PYTHON=$(command -v python3)
if [ ! "${PYTHON}" ]; then
  PYTHON=$(command -v python)
fi

if [ "${PYTHON}" ]; then
  HERE=$(cd "$(dirname "$0")" && pwd -P)
  NAME=$(basename "$0" .sh)
  SCRT=${NAME}.py

  if [ "${HERE}" ]; then SCRT_A=${HERE}/${SCRT}; fi
  if [ "${LIBXSROOT}" ]; then SCRT_B=${LIBXSROOT}/scripts/${SCRT}; fi
  if [ "${REPOREMOTE}" ]; then SCRT_C=${REPOREMOTE}/libxs/scripts/${SCRT}; fi
```

```
  if [ "${SCRT_A}" ] && [ -e "${SCRT_A}" ]; then
    ${PYTHON} "${SCRT_A}" --token "${TOKEN}" "$@"
  elif [ "${SCRT_B}" ] && [ -e "${SCRT_B}" ]; then
    ${PYTHON} "${SCRT_B}" --token "${TOKEN}" "$@"
  elif [ "${SCRT_C}" ] && [ -e "${SCRT_C}" ]; then
    ${PYTHON} "${SCRT_C}" --token "${TOKEN}" "$@"
  else
    >&2 echo -n "ERROR: missing ${SCRT_A}"
    if [ "${SCRT_B}" ]; then >&2 echo -n " or ${SCRT_B}"; fi
    if [ "${SCRT_C}" ]; then >&2 echo -n " or ${SCRT_C}"; fi
    >&2 echo "!"
    exit 1
  fi
else
  >&2 echo "ERROR: missing prerequisites!"
  exit 1
fi
```

The following flow is established:

1. Connect to a specified pipeline (online) or load a logfile directly (offline).
2. Populate an entry (JSON-block or telegram) under a "build number", "category", and "case".
3. Plot "execution time" over the history of build numbers.

There are several command-line options to customize each of the above steps (`--help` or `-h`):

- To only plot data (already collected), use `-i ""` to omit a network connection.
- To query, e.g., ResNet-50 results, use `-y resnet-50` (case-insensitive).
- Multiple results can be combined, i.e., use `-y` (space-separated words).
- To exactly match (single results) use `-x` in addition to `-y`.
- To limit and select a specific "category" (instead of all), use `-s`.
- Select exactly using `-z`, e.g., `-z -s "clx"` (omits, e.g., "clx").
- Create a PDF (vector graphics have infinite resolution), use `-g myreport.pdf`.
- Adjust pixel resolution, aspect ratio, or density, use `-d 1200x800`.

The level of verbosity (`-v`) can be adjusted (0: quiet, 1: automation, 2: progress). Default verbosity shows progress (downloading results) whereas "automation" allows to further automate reports, e.g., get the filename of the generated plot (errors are generally printed to `stderr`). Loading a logfile into the database directly can serve two purposes: (1) debugging the supported format like "telegram" or JSON, and (2) offline operation. The latter can be also useful if for instance a CI-agents produces a log, i.e., it can load into the database right away. Command-line options also allow for "exact placement" (`-j`) by specifying the build number supposed to take the loaded data (data is appended by default, i.e., it is assumed to be a new build, or the build number is incremented).

The tool automatically performs database backups of the historic values (`-n`) according to the retention (`-k`). The retention can also be used to prune the history (by forcing an earlier backup). Backups carry a timestamp as part of the filename (database), and contain the full history of value at the time of the backup.

Examples (omit `-i ""` if downloading results is desired):

- Plot ResNet-50 results from CI-pipeline "tpp-libxs" for "clx" systems:
  `scripts/tool_report.sh -p tpp-libxs -i "" -y "resnet-50" -z -s "clx"`.
- Like above request, but only FP32 results:
  `scripts/tool_report.sh -p tpp-libxs -i "" -x -y "ResNet-50 (fwd, mb=1, f32)" -z -s "clx"`.
- Like above request, but alternatively ("all" operator is also default):
  `scripts/tool_report.sh -p tpp-libxs -i "" -u "all" -y "resnet f32" -z -s "clx"`.
- Plot ResNet-50 results from CI-pipeline "tpp-plaidml":
  `scripts/tool_report.sh -p tpp-plaidml -i "" -r "duration_per_example,1000,ms"`
- Plot "GFLOP/s" for "conv2d_odd_med" from CI-pipeline "tpp-plaidml":
  `scripts/tool_report.sh -p tpp-plaidml -i "" -y "conv2d_odd_med" -r "gflop"`
- Plot "tpp-benchmark" pipeline (MLIR benchmarks, main-branch):
  `scripts/tool_report.sh -p tpp-benchmark -i "" -y "" -r "mlir" -b "main"`
- Plot "tpp-benchmark" pipeline (MLIR benchmarks, main-branch, untied plots):
  `scripts/tool_report.sh -p tpp-benchmark -i "" -y "" -r "mlir" -b "main" -u`

- Plot "tpp-benchmark" pipeline (reference benchmarks; selected entries `-y`):
  `scripts/tool_report.sh -p tpp-benchmark -i "" -q "any" -y "gemm matmul" -r "dnn"`
- Plot "tpp-benchmark" pipeline (MLIR benchmarks without "single", untied plots):
  `scripts/tool_report.sh -p tpp-benchmark -i "" -q "not" -y "single" -r "mlir" -u`
- Plot "tpp-performance" pipeline (MLIR benchmarks only "mlp", untied plots):
  `scripts/tool_report.sh -p tpp-performance -i "" -y "mlp" -r "mlir" -u`

The exit code of the script is non-zero in case of an error, or if the latest value deviates and exceeds the margin (`--bounds`). For the latter, the meaning of the values must be given (like "higher is better"). The first argument of the bounds is a factor such that the standard deviation of historic values is amplified to act as margin of the relative deviation (latest versus previous value). The second argument of the bounds determines the accepted percentage of deviation (latest versus previous value).

The exit code is only impacted if an explicit sign is given to determine "bad" values (`+` or `-`). For example, `2.0` gives a factor of two over standard deviation (no impact for the exit code), `2.0 10` is likewise but also limits the deviation to 10% at most, `+3.0` gives a factor of three over standard deviation and considers a positive deviation as regression (like for timing values) and thereby impacts the exit code. Also, it is possible to just determine the meaning and keep default bounds, .i.e., only `-` determines negative deviation as a regression (like for higher-is-better values).

By default, only the last/current vs the previous value (history) determines the deviation. This can miss a historic slow/steady regession if the last value is always below threashold. To consider the full history value (median) as part of the deviation, a leading `0` can be given in front of the afore mentioned factor (after the sign). For example, `-02.1` specifies an aplification factor of 2.1 over standard deviation with deviation considering the median of all historic values. Consequently, a fractional factor requires two leading zeros, e.g., `-00.4`.